

# REGRESSION MODEL COMPARISON – R<sup>2</sup> SCORE TEMPLATE

Insurance Charges Prediction • Multiple Regression Models • Hyperparameter Tuning

## 1. DATASET / EXPERIMENT DETAILS

| ITEM                     | DETAILS                                                                   |
|--------------------------|---------------------------------------------------------------------------|
| Dataset name             | insurance_pre.csv                                                         |
| Target variable (y)      | charges                                                                   |
| Input features (X)       | age, bmi, children, sex_male, smoker_yes                                  |
| Total rows / columns     | 1338 rows / 6 columns                                                     |
| Train / Test split       | 70% training, 30% testing                                                 |
| Random state             | 0                                                                         |
| Pre-processing / Scaling | sex and smoker → one-hot encoding (drop_first=True); scaling only for SVR |

## 2. MULTIPLE LINEAR REGRESSION (LinearRegression)

R<sup>2</sup> value: 0.789479

Best configuration: fit\_intercept = True | positive = False

| S.NO | fit_intercept | positive | tol  | R <sup>2</sup> VALUE |
|------|---------------|----------|------|----------------------|
| 1    | True          | False    | 1e-6 | <b>0.789479</b>      |
| 2    | False         | False    | 1e-6 | 0.762372             |
| 3    | True          | True     | 1e-6 | 0.789443             |
| 4    | False         | True     | 1e-6 | 0.761433             |

## 3. SUPPORT VECTOR REGRESSION (SVR)

R<sup>2</sup> value: 0.873373

Best configuration: Kernel = RBF | C = 3000 | gamma = 0.1 | epsilon = 0.5 | degree = 3 | coef0 = 1.0

| S.NO | C     | LINEAR (R <sup>2</sup> ) | RBF (R <sup>2</sup> ) | POLY (R <sup>2</sup> ) | SIGMOID (R <sup>2</sup> ) |
|------|-------|--------------------------|-----------------------|------------------------|---------------------------|
| 1    | C10   | 0.462468                 | -0.032273             | 0.038716               | 0.039307                  |
| 2    | C100  | 0.628879                 | 0.320032              | 0.617957               | 0.527610                  |
| 3    | C500  | 0.763106                 | 0.664298              | 0.826368               | 0.444606                  |
| 4    | C1000 | 0.764931                 | 0.810206              | 0.856649               | 0.287471                  |
| 5    | C2000 | 0.744042                 | 0.854777              | 0.860558               | -0.593951                 |
| 6    | C3000 | 0.741424                 | <b>0.866339</b>       | 0.859893               | -2.124419                 |

### 3.1 SVR – Secondary Hyperparameter Tuning

| PARAMETER | VALUES TO TEST                   | APPLIES TO           | BEST VALUE | BEST R <sup>2</sup> |
|-----------|----------------------------------|----------------------|------------|---------------------|
| gamma     | scale, auto, 0.001, 0.01, 0.1, 1 | rbf / poly / sigmoid | 0.1        | 0.868695            |
| epsilon   | 0.01, 0.05, 0.1, 0.2, 0.5        | all kernels          | 0.50       | 0.868696            |
| degree    | 2, 3, 4, 5                       | poly only            | 3          | 0.808619            |
| coef0     | 0, 0.1, 0.5, 1                   | poly / sigmoid       | 1.0        | <b>0.873373</b>     |

## 4. DECISION TREE REGRESSION (DecisionTreeRegressor)

R<sup>2</sup> value: 0.863731

**Best configuration:** Criterion = poisson | max\_features = sqrt | splitter = best | max\_depth = 8 | min\_samples\_split = 2 | min\_samples\_leaf = 10 | max\_leaf\_nodes = None | min\_impurity\_decrease = 0 | ccp\_alpha = 0

| S.NO | CRITERION      | MAX FEATURES | SPLITTER | R <sup>2</sup> VALUE |
|------|----------------|--------------|----------|----------------------|
| 1    | squared_error  | None         | best     | 0.695710             |
| 2    | squared_error  | None         | random   | 0.721020             |
| 3    | squared_error  | sqrt         | best     | 0.675469             |
| 4    | squared_error  | sqrt         | random   | 0.696930             |
| 5    | squared_error  | log2         | best     | 0.675469             |
| 6    | squared_error  | log2         | random   | 0.696930             |
| 7    | absolute_error | None         | best     | 0.647619             |
| 8    | absolute_error | None         | random   | 0.718046             |
| 9    | absolute_error | sqrt         | best     | <b>0.722524</b>      |
| 10   | absolute_error | sqrt         | random   | 0.711573             |
| 11   | absolute_error | log2         | best     | <b>0.722524</b>      |
| 12   | absolute_error | log2         | random   | 0.711573             |
| 13   | poisson        | None         | best     | 0.728816             |
| 14   | poisson        | None         | random   | 0.701949             |
| 15   | poisson        | sqrt         | best     | <b>0.755269</b>      |
| 16   | poisson        | sqrt         | random   | 0.626101             |
| 17   | poisson        | log2         | best     | <b>0.755269</b>      |
| 18   | poisson        | log2         | random   | 0.626101             |

#### 4.1 Decision Tree – Additional Hyperparameters

| PARAMETER             | VALUES TO TEST           | PURPOSE                            | BEST VALUE       | BEST R <sup>2</sup> |
|-----------------------|--------------------------|------------------------------------|------------------|---------------------|
| max_depth             | None, 2, 3, 5, 8, 10, 15 | Control tree depth / overfitting   | 8                | 0.826557            |
| min_samples_split     | 2, 5, 10, 20             | Minimum samples needed to split    | 2                | 0.826557            |
| min_samples_leaf      | 1, 2, 5, 10              | Minimum samples in a leaf          | 10               | <b>0.863731</b>     |
| max_leaf_nodes        | None, 5, 10, 20, 50      | Limit number of leaves             | None             | <b>0.863731</b>     |
| min_impurity_decrease | 0, 0.001, 0.01, 0.05     | Require minimum impurity reduction | 0                | <b>0.863731</b>     |
| ccp_alpha             | 0, 0.001, 0.01, 0.05     | Cost-complexity pruning            | 0 / 0.001 / 0.01 | <b>0.863731</b>     |

### 5. RANDOM FOREST REGRESSION (RandomForestRegressor)

R<sup>2</sup> value: **0.885356**

**Best configuration:** Criterion = squared\_error | max\_features = sqrt | n\_estimators = 100 | max\_depth = 8 | min\_samples\_split = 2 | min\_samples\_leaf = 1 | max\_leaf\_nodes = None | min\_impurity\_decrease = 0.05 | ccp\_alpha = 0 | bootstrap = True

| S.NO | Criterion            | Max Features | N Estimators | R <sup>2</sup>  |
|------|----------------------|--------------|--------------|-----------------|
| 1    | MSE (squared_error)  | 10           | 10           | 0.833786        |
| 2    | MSE (squared_error)  | 10           | 100          | 0.853978        |
| 3    | MSE (squared_error)  | 100          | 10           | 0.833786        |
| 4    | MSE (squared_error)  | 100          | 100          | 0.853978        |
| 5    | MSE (squared_error)  | sqrt         | 10           | 0.851654        |
| 6    | MSE (squared_error)  | sqrt         | 100          | <b>0.871100</b> |
| 7    | MSE (squared_error)  | log2         | 10           | 0.851654        |
| 8    | MSE (squared_error)  | log2         | 100          | <b>0.871100</b> |
| 9    | MAE (absolute_error) | 10           | 10           | 0.836387        |
| 10   | MAE (absolute_error) | 10           | 100          | 0.849802        |
| 11   | MAE (absolute_error) | 100          | 10           | 0.836387        |
| 12   | MAE (absolute_error) | 100          | 100          | 0.849802        |
| 13   | MAE (absolute_error) | sqrt         | 10           | 0.857975        |
| 14   | MAE (absolute_error) | sqrt         | 100          | 0.869248        |
| 15   | MAE (absolute_error) | log2         | 10           | 0.857975        |
| 16   | MAE (absolute_error) | log2         | 100          | 0.869248        |

## 5.1 Random Forest – Additional Hyperparameters

| PARAMETER             | VALUES TO TEST        | PURPOSE                            | BEST VALUE | BEST R <sup>2</sup> |
|-----------------------|-----------------------|------------------------------------|------------|---------------------|
| max_depth             | None, 3, 5, 8, 10, 15 | Control tree depth / overfitting   | 8          | 0.885274            |
| min_samples_split     | 2, 5, 10, 20          | Minimum samples needed to split    | 2          | 0.885274            |
| min_samples_leaf      | 1, 2, 5, 10           | Minimum samples in a leaf          | 1          | 0.885274            |
| max_leaf_nodes        | None, 10, 20, 50, 100 | Limit number of leaves             | None       | 0.885274            |
| min_impurity_decrease | 0, 0.001, 0.01, 0.05  | Require minimum impurity reduction | 0.05       | 0.885356            |
| bootstrap             | True, False           | Use bootstrap samples              | True       | 0.885356            |
| ccp_alpha             | 0, 0.001, 0.01, 0.05  | Cost-complexity pruning            | All tied   | 0.885356            |

## 6. FINAL MODEL COMPARISON

| MODEL                      | BEST HYPERPARAMETERS                                                                                                                                                                             | R <sup>2</sup> SCORE | RANK | REMARKS                       |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|------|-------------------------------|
| Multiple Linear Regression | fit_intercept=True, positive=False                                                                                                                                                               | 0.789479             | 4    | Baseline linear model         |
| SVR                        | kernel=poly, C=3000, gamma=0.1, epsilon=0.5, degree=3, coef0=1.0                                                                                                                                 | 0.873373             | 2    | Good predictive performance   |
| Decision Tree Regression   | criterion=poisson, max_features=sqrt, splitter=best, max_depth=8, min_samples_split=2, min_samples_leaf=10, max_leaf_nodes=None, min_impurity_decrease=0, ccp_alpha=0                            | 0.863731             | 3    | Good performance after tuning |
| Random Forest Regression   | criterion=squared_error, max_features=sqrt, n_estimators=100, max_depth=8, min_samples_split=2, min_samples_leaf=1, max_leaf_nodes=None, min_impurity_decrease=0.05, ccp_alpha=0, bootstrap=True | 0.885356             | 1    | Best-performing model         |

Best overall model based on R<sup>2</sup>: **Random Forest Regression**

Highest R<sup>2</sup> score: **0.885356**

## 7. FINAL NOTES

The objective of this assignment was to develop regression models to predict insurance charges using the insurance dataset. Four algorithms were implemented and evaluated: Multiple Linear Regression, Support Vector Regression (SVR), Decision Tree Regression, and Random Forest Regression. Their performance was compared using the R<sup>2</sup> score.

Among the four models, Random Forest Regression achieved the highest R<sup>2</sup> score of **0.885356**, followed by SVR with 0.873373, Decision Tree Regression with 0.863731, and Multiple Linear Regression with 0.789479.

Therefore, **Random Forest Regression is the best-performing model among the tested configurations** for predicting insurance charges in this experiment. It explains approximately 88.54% of the variation in the test-set insurance charges, based on the R<sup>2</sup> score.

## 8. OFFICIAL scikit-learn REFERENCES

LinearRegression: [https://scikit-learn.org/stable/modules/generated/sklearn.linear\\_model.LinearRegression.html](https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html)

SVR: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVR.html>

DecisionTreeRegressor: <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeRegressor.html>

RandomForestRegressor: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>